

SIMATS ENGINEERING
ASSESSMENT TOOL 1: Algorithm Design Challenge

COURSE NAME: **ARTIFICIAL
INTELLIGENCE
AND EXPERT SYSTEMS**

COURSE CODE: **MLA01**

COURSE OUTCOME COVERED

CO1: *Apply search algorithms and heuristic techniques to solve state-space search and optimization problems. (BTL 3)*

Assessment Tool Weightage: 40 %

Total Marks: 100

Time: 90 Mins

No. of Questions: 5

Annexure A
Algorithm Design Challenge

Code of Conduct:

I P Surendra Kumar Reddy (192425224) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Criterion	Weightage	Marks Obtained
Problem Analysis and Understanding	15%	
AI Algorithm Selection & Justification	15%	
Algorithm Design / Pseudocode	25%	
Application of AI Concepts (Greedy, A*, CSP, Minimax, etc.)	15%	
Diagram / State Space / Search Tree Representation	10%	
Complexity Analysis and Solution Efficiency	10%	
Originality, Critical Thinking, and Presentation	10%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

Answer all the Questions

Q.No	Question	Topics
1	Design an algorithm using Greedy Best-First Search to find a path from a source node to a goal node for the given Romania Map Problem graph. Clearly define the heuristic function, show the node expansion order, and justify why Greedy Best-First Search is suitable for this problem.	Informed Search, Greedy Best-First Search, Heuristic Functions
2	A delivery robot must travel from a warehouse to a customer through a weighted road network. Design an A Search algorithm to determine the optimal path. Calculate $g(n)$, $h(n)$, and $f(n)$ for each expanded node, and explain why A guarantees the optimal solution.	A* Search, Heuristic Functions
3	A university needs to prepare an examination timetable without scheduling conflicting subjects at the same time. Design the problem as a Constraint Satisfaction Problem (CSP) by identifying variables, domains, and constraints. Explain how MRV and Forward Checking improve the efficiency of the solution.	CSP, MRV, Forward Checking
4	Design the Minimax algorithm for the given game tree to determine the best move for the MAX player. Then apply Alpha-Beta Pruning to the same tree and indicate which nodes are pruned. Compare the number of nodes evaluated before and after pruning.	Game Playing, Minimax, Alpha-Beta Pruning
5	A warehouse robot operates in a dynamic environment where obstacles may appear unexpectedly. Design an intelligent search strategy using either Local Search or an Online Search Agent. Explain how the algorithm adapts to environmental changes and compare its performance with informed search algorithms.	Local Search, Optimization, Online Search Agents

Structure of the Answer – Algorithm Design Challenge

S.No	Sections	Expected Content
1	Problem Statement	Briefly describe the given problem in your own words.
2	Problem Analysis	Identify the initial state, goal state, assumptions, and constraints.
3	AI Technique Selection	Specify the most appropriate AI algorithm/search technique (e.g., Greedy Best-First Search, A*, CSP, Minimax, Local Search).
4	Justification of Algorithm	Explain why the selected algorithm is suitable compared to other search techniques.
5	State Space Representation	Represent the problem using a state-space diagram, search graph, game tree, or constraint graph, as applicable.
6	Variables, Domains, and Constraints (Applicable for CSP problems)	Identify variables, possible domains, and constraints. If not applicable, mention "Not Applicable".
7	Heuristic Function (Applicable for Informed Search)	Define the heuristic function $h(n)$ and explain whether it is admissible or appropriate for the problem. If not applicable, mention "Not Applicable".
8	Algorithm Design / Pseudocode	Write the step-by-step algorithm or pseudocode for solving the problem.
9	Flowchart / Algorithm Diagram	Draw a flowchart, search tree, game tree, or algorithm workflow illustrating the solution.
10	Working / Execution Trace	Demonstrate the execution of the algorithm with at least one example, showing intermediate steps such as node expansion, constraint propagation, or game-tree evaluation.
11	Complexity Analysis	Analyze the Time Complexity and Space Complexity of the proposed algorithm.
12	Advantages and Limitations	Discuss the strengths and limitations of the selected algorithm for the given problem.
13	Conclusion	Summarize the effectiveness of the designed algorithm and suggest possible improvements or alternative approaches.

Q1. GREEDY BEST-FIRST SEARCH – ROMANIA MAP

1. Problem Statement & Analysis

Find a path from the **source node to the goal node** in the Romania Map using Greedy Best-First Search. The algorithm selects the node that appears closest to the goal.

- **Initial State:** Source node
- **Goal State:** Goal node
- **Output:** Path from source to goal

2. AI Technique & Justification

Greedy Best-First Search (GBFS) is an informed search algorithm. It selects the node with the **lowest heuristic value**, making it suitable for quickly reaching the goal.

3. Heuristic Function

$h(n)$ = straight-line distance from node to Bucharest. The node with the lowest $h(n)$ is expanded first.

4. Pseudocode

1. $OPEN \leftarrow \{\text{Start}\}$, $CLOSED \leftarrow \emptyset$
2. While $OPEN$ is not empty:
3. Select node n with minimum $h(n)$
4. If $n = \text{Goal}$, return path
5. Move n to $CLOSED$
6. Generate successors of n
7. Add unvisited successors to $OPEN$
8. Return FAILURE

5. Node Expansion

Expansion Order: Select and expand the node with the lowest $h(n)$ until the goal is reached.

6. Complexity

Time: $O(b^m)$

Space: $O(b^m)$

7. Advantages & Limitations

Advantages: Fast, simple, uses heuristic guidance.

Limitations: Does not consider path cost and may not give the optimal path.

8. Conclusion

Greedy Best-First Search is suitable for the Romania Map because the heuristic guides the search toward the goal efficiently.

Q1. GREEDY BEST-FIRST SEARCH (ROMANIA MAP) FLOWCHART

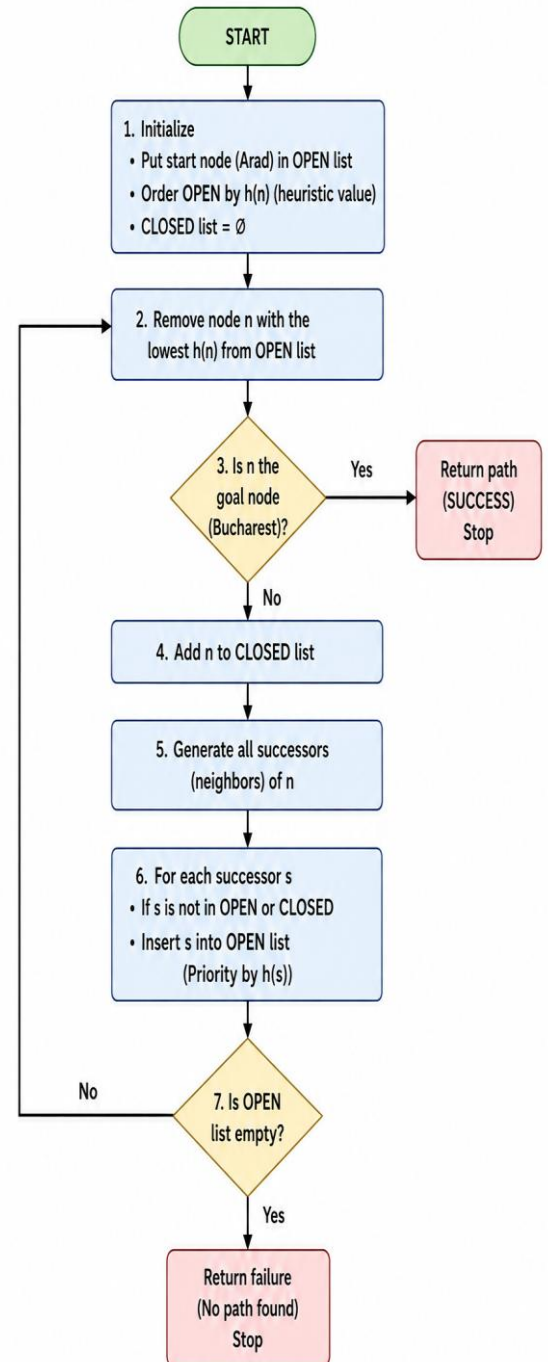


Figure 1: Greedy Best-First Search Flowchart

Q2. A* Search – Warehouse-to-Customer Delivery Robot

1. Problem Statement & Analysis

A delivery robot must find the **least-cost path** from the **Warehouse (S)** to the **Customer (G)** through a weighted road network.

2. AI Technique & Justification

A* Search combines actual and estimated cost:

$$f(n) = g(n) + h(n)$$

It is suitable because it considers both **path cost** and **distance to the goal**.

3. Heuristic Function

$$h(n) = \text{estimated cost from } n \text{ to Goal}$$

A straight-line distance can be used as an admissible heuristic.

4. Algorithm / Pseudocode

1. Initialize OPEN with Start and CLOSED as empty.
2. Select the node with minimum **f(n)**.
3. Expand its successors and calculate **g(n)**, **h(n)**, **f(n)**.
4. Repeat until the Goal is reached.

5. Trace

Node	g(n)	h(n)	f(n)
S	0	h(S)	g(S)+h(S)
A	g(A)	h(A)	g(A)+h(A)
G	g(G)	0	g(G)

6. Complexity

Time: $O(b^d)$

Space: $O(b^d)$

7. Why A* Guarantees Optimality

A* gives the optimal path when **h(n)** is admissible, meaning it never overestimates the remaining cost.

8. Conclusion

A* efficiently finds the **minimum-cost route** by combining actual path cost with heuristic estimation.

A* Search – Flowchart

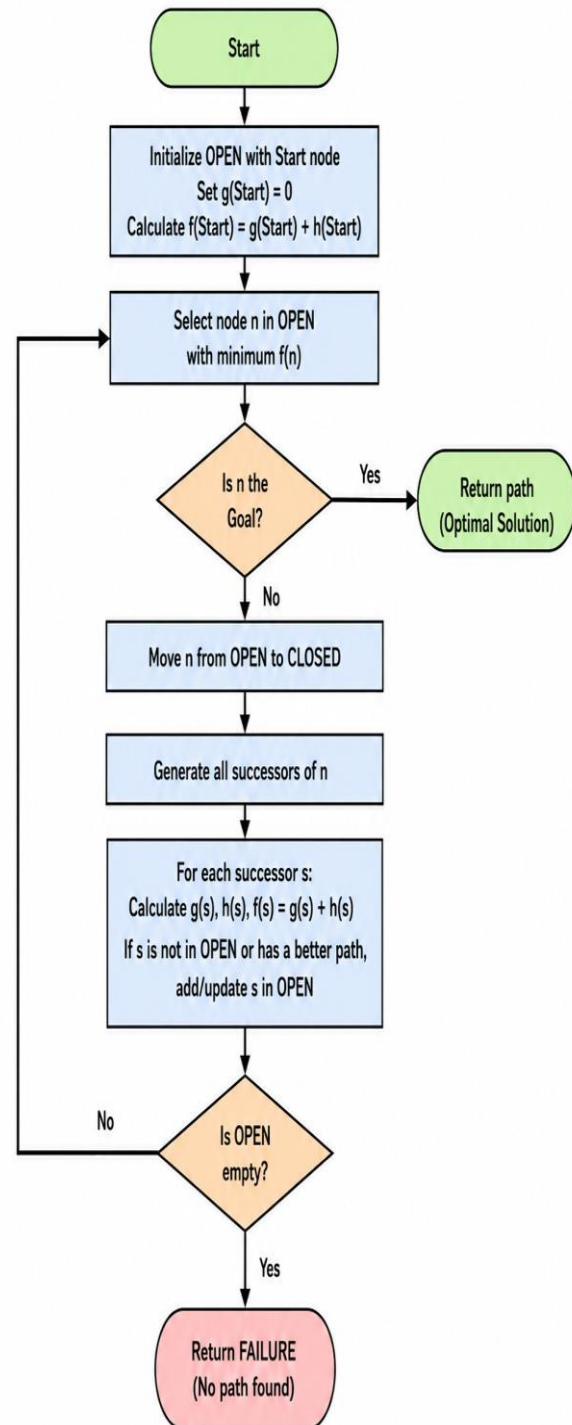


Figure 2: A* Search Flowchart.

Q3. CSP – University Examination Timetable

1. Problem Statement & Analysis

Schedule all subjects into exam slots without placing **conflicting subjects at the same time**.

- **Variables:** Subjects
- **Domains:** Available exam time slots
- **Constraint:** Conflicting subjects must have different slots.

2. AI Technique & Justification

Constraint Satisfaction Problem (CSP) is suitable because the timetable can be solved by assigning a valid slot to every subject while satisfying all constraints.

3. CSP Representation

Variables + Domains + Constraints

Ex: **Subjects:** A, B, C, D
Slots: S1, S2, S3

4. MRV

Minimum Remaining Values (MRV) selects the subject having the **fewest available slots** first, reducing unnecessary search.

5. Forward Checking

After assigning a slot, **Forward Checking** removes that slot from **conflicting subjects**, detecting conflicts early.

6. Algorithm

1. Select unassigned subject using MRV
2. Assign a valid time slot
3. Apply Forward Checking
4. If conflict occurs, backtrack
5. Repeat until all subjects are assigned

7. Complexity

Time: $O(d^n)$

Space: $O(n)$

8. Conclusion

CSP with **MRV and Forward Checking** efficiently produces a valid examination timetable while reducing unnecessary assignments.

Figure 3: CSP – University Examination Timetable (Flowchart)

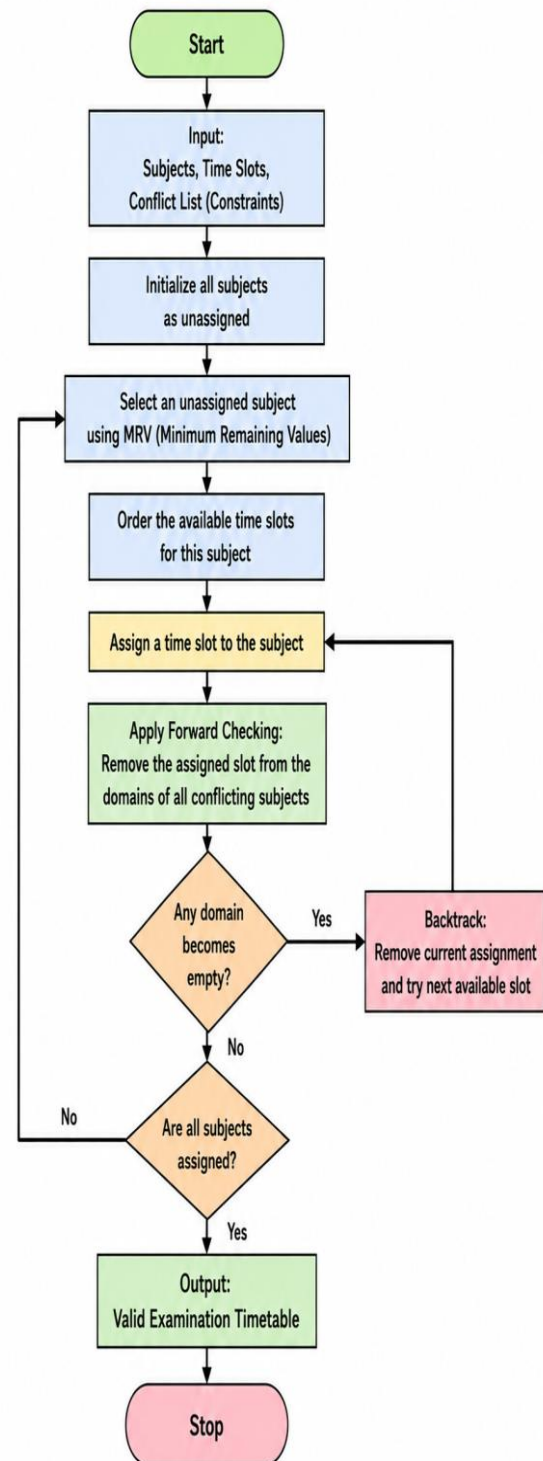


Figure 3: CSP – University Examination Timetable Flowchart

Q4. Minimax & Alpha-Beta Pruning

1. Problem Statement & Analysis

Find the **best move for the MAX player** using the given game tree.

- **MAX:** Maximizes the score
- **MIN:** Minimizes the score
- **Output:** Best move for MAX

2. Minimax Algorithm

Minimax evaluates the game tree recursively and selects the **maximum value at MAX nodes** and **minimum value at MIN nodes**.

3. Algorithm / Pseudocode

MINIMAX (node):

If node is terminal, return utility(node)

If MAX node → return maximum child value

If MIN node → return minimum child value

4. Alpha-Beta Pruning

Alpha-Beta pruning improves Minimax by **ignoring branches that cannot affect the final decision**.

- **α (Alpha):** Best value for MAX
- **β (Beta):** Best value for MIN
- Prune when $\alpha \geq \beta$

5. Execution

Execution: Evaluate the game tree, apply Minimax, and prune branches when $\alpha \geq \beta$.

6. Complexity

- **Minimax:** $O(b^d)$
- **Alpha-Beta:** $O(b^{(d/2)})$ with good ordering

7. Conclusion

Alpha-Beta pruning gives the **same optimal move as Minimax** while evaluating fewer nodes.

Minimax & Alpha-Beta Pruning Flowchart

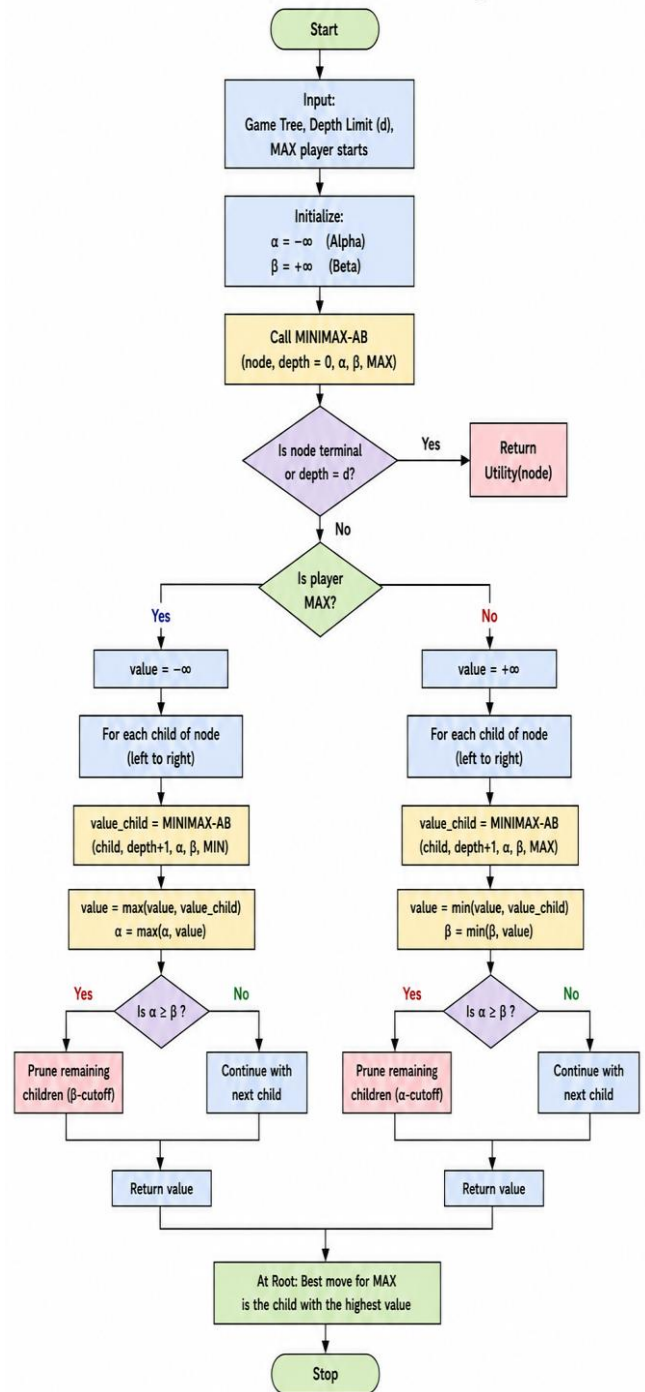


Figure 4: Minimax & Alpha-Beta Pruning Flowchart

Q5. Online Search Agent – Warehouse Robot

1. Problem Statement & Analysis

A warehouse robot must reach a target while obstacles may appear unexpectedly.

- Initial State: Robot position
- Goal State: Target position
- Constraint: Dynamic obstacles

2. AI Technique & Justification

Online Search Agent is suitable because it **observes the environment and replans the path** when obstacles appear.

3. Algorithm / Pseudocode

```
Start
Observe environment
Select best available move
Move to next state
If obstacle appears:
    Update path
    Replan
Repeat until goal is reached
Stop
```

4. Working

The robot **senses** → **moves** → **detects obstacles** → **replans**, adapting to environmental changes.

5. Comparison

- Online Search: Adapts to unknown/dynamic changes.
- Informed Search: Uses available heuristic information for path planning.

6. Complexity

Time: $O(b^d)$

Space: $O(d)$

7. Advantages & Limitation

Advantage: Adapts to changing environments.

Limitation: Replanning may increase computation.

8. Conclusion

Online Search is effective for warehouse robots operating in **dynamic environments**.

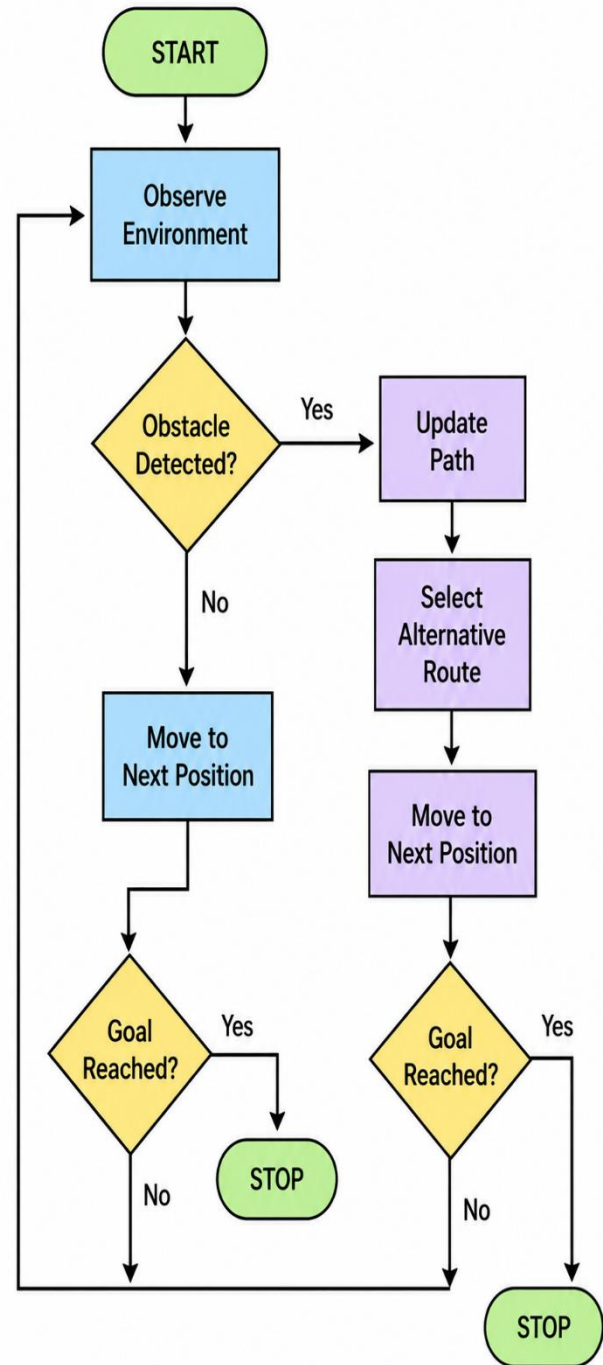


Figure 5: Online Search Agent for a Warehouse Robot Flowchart

EVALUATION RUBRICS

Criteria	Weightage (%)	Excellent (90–100%)	Good (75–89%)	Average (60–74%)	Poor (<60%)
Problem Analysis and Understanding	15%	13–15% – Clearly identifies the problem, initial state, goal state, constraints, and expected outcome.	10–12% – Identifies most problem components with minor omissions.	7–9% – Demonstrates partial understanding with some missing elements.	0–6% – Incorrect or incomplete understanding of the problem.
AI Algorithm Selection & Justification	15%	13–15% – Selects the most appropriate AI algorithm/search technique with strong technical justification.	10–12% – Selects an appropriate algorithm with adequate justification.	7–9% – Selects a partially suitable algorithm with limited justification.	0–6% – Incorrect algorithm selection or no justification.
Algorithm Design / Pseudocode	25%	22–25% – Designs a complete, logical, efficient, and error-free algorithm/pseudocode.	17–21% – Algorithm is mostly correct with minor logical errors.	12–16% – Algorithm is partially complete with several logical errors.	0–11% – Algorithm is incomplete, incorrect, or difficult to understand.
Application of AI Concepts (Greedy, A*, CSP, Minimax, Alpha-Beta, etc.)	15%	13–15% – Correctly applies the relevant AI concepts and techniques throughout the solution.	10–12% – Applies AI concepts correctly with minor mistakes.	7–9% – Demonstrates limited application of AI concepts.	0–6% – Unable to apply the appropriate AI concepts or techniques.

Diagram / State Space / Search Tree Representation	10%	9–10% – Accurate, complete, and well-labelled diagram supporting the solution.	7–8% – Diagram is mostly correct with minor omissions.	5–6% – Diagram is partially complete or lacks clarity.	0–4% – Diagram is missing or incorrect.
Complexity Analysis and Solution Efficiency	10%	9–10% – Correctly analyzes time and space complexity and proposes an efficient solution.	7–8% – Complexity analysis is mostly correct with minor errors.	5–6% – Partial complexity analysis or limited discussion of efficiency.	0–4% – No meaningful complexity analysis or highly inefficient solution.
Originality, Critical Thinking, and Presentation	10%	9–10% – Demonstrates excellent originality, logical reasoning, organized presentation, and explores optimized/alternative solutions.	7–8% – Demonstrates good critical thinking with a clear presentation.	5–6% – Shows limited originality and acceptable presentation.	0–4% – Poor presentation with little or no critical thinking.